

CURSO DE INTELIGENCIA ARTIFICIAL PARA PRINCIPIANTES

MÓDULO 1

El Lenguaje de las Máquinas

Cómo una IA transforma tus palabras en números, relaciona significados y construye una respuesta paso a paso



Edición para bachillerato y primer semestre · Ingeniería de Software · Requiere: ideas básicas del Módulo 0

★ **Idea central del módulo:** los *embeddings* permiten representar significados con números. Las demás secciones te preparan para comprender esta idea sin matemáticas avanzadas.

SECCIÓN

0

Antes de empezar*Qué aprenderás y cómo aprovechar este tutorial***La gran pregunta de este módulo**

En el Módulo 0 viste que un modelo de IA guarda patrones aprendidos en una enorme colección de números llamados **pesos**. Pero aparece una pregunta: si la máquina trabaja con matemáticas, **¿cómo puede recibir una frase como “hola, ¿me ayudas?” y producir una respuesta que tenga sentido?**

Para responderla recorrerás **cuatro estaciones**. En cada una aprenderás una idea, verás un ejemplo y realizarás una actividad corta. No necesitas matemáticas avanzadas: basta con leer con calma y seguir la frase del gato.

Estación	En palabras simples	Sección
 Tokens	La IA divide el texto en piezas pequeñas. Esas piezas ocupan espacio y pueden generar costo.	1.1
 Embeddings	El significado se representa con coordenadas. Los textos parecidos quedan cerca en ese mapa.	1.2
 Atención	Cada pieza usa el contexto de las demás y el modelo calcula cuál podría venir después.	1.3
 Prompts	Usas estas ideas para escribir instrucciones claras, comprobables y fáciles de reutilizar.	1.4

 Cómo leer las cajas de colores

 **Concepto clave** (azul: comprende la idea) ·  **Ejemplo** (morado: observa cómo funciona) ·  **Tip práctico** (amarillo: aplícalo) ·  **Cuidado** (rojo: evita este error) ·  **Mini-actividad** (verde azulado: comprueba lo aprendido) ·  **Reto** (verde: producto para entregar) ·  **Idea para llevar** (dorado: lo esencial).

 Hilo conductor: la frase del gato

Usaremos la misma frase durante todo el módulo: **“El gato se subió al tejado”**. Primero veremos cómo puede dividirse en piezas (1.1); después, cómo se representa su significado (1.2); luego, cómo el contexto relaciona sus palabras (1.3); y finalmente, cómo incluirla en una instrucción bien escrita (1.4). Si algo parece difícil, vuelve a esta frase.

Al terminar este módulo podrás...

- ▶ Hacer una **estimación** de los tokens de un texto y calcular un costo aproximado de uso por API.
- ▶ Explicar un **embedding** con la analogía del mapa y comparar dos significados mediante su cercanía.
- ▶ Explicar, con tus palabras, cómo el contexto ayuda al modelo y cómo se genera una respuesta pieza por pieza.
- ▶ Escribir **prompts claros** con contexto, ejemplos, reglas y una salida estructurada en JSON.

SECCIÓN

1.1

Tokens: cómo lee la IA

Cómo se divide el texto y por qué importa en una aplicación



Brújula de la sección — al terminar sabrás:

- ▶ Qué es un token mediante la analogía de las piezas de LEGO.
- ▶ Por qué un modelo puede equivocarse al contar letras o manipular caracteres.
- ▶ Qué es la ventana de contexto y por qué una conversación muy larga puede perder información.
- ▶ Cómo estimar el costo de una consulta a una API con unas pocas líneas de Python.

Primera sorpresa: un modelo de lenguaje no recibe el texto exactamente como tú lo ves. Antes de procesarlo, **un tokenizador lo divide en piezas llamadas tokens**. Saber esto te ayuda a entender tres cosas: cuánto texto cabe en una consulta, por qué algunas tareas con letras son difíciles y cuánto puede costar usar una IA desde un programa.



¿Qué es un token? Imagina piezas de LEGO 🧱

Un **token** es una pieza de texto que el modelo procesa. Puede ser una palabra completa, una parte de una palabra, un número, un signo o incluso una combinación que incluye un espacio. Por eso **token** y **palabra** no significan lo mismo.

Cada modelo posee un **vocabulario de tokens** definido antes del entrenamiento. El tokenizador combina esas piezas para representar textos de muchos idiomas. Las secuencias frecuentes suelen ocupar pocas piezas; las palabras poco comunes, los nombres nuevos o los errores de escritura pueden dividirse en varias. Existen distintos métodos de tokenización, como BPE, pero por ahora solo necesitas comprender el resultado.

Nuestra frase, dividida en piezas

Un tokenizador **podría** dividir “El gato se subió al tejado” como se muestra abajo. El punto · representa un espacio unido a la pieza siguiente:

El	·gato	·se	·sub	ió	·al	·tej	ado
1	2	3	4	5	6	7	8

Observa dos ideas importantes:

- ▶ En este ejemplo, “subió” y “tejado” se dividen. Otro modelo podría cortarlas de manera diferente, porque cada tokenizador tiene su propio vocabulario.
- ▶ Aquí vemos 6 palabras y 8 tokens. Es un **ejemplo ilustrativo**, no una regla universal. Para conocer el número real de una aplicación, usa el contador del modelo o de su proveedor.

Error típico: pedir exactitud con letras y confiar sin comprobar

El modelo puede recibir “fresa” como una sola pieza o como varias, en lugar de trabajar letra por letra desde el inicio. Por eso puede fallar al contar caracteres, ordenar letras o escribir una palabra al revés. Cuando necesites exactitud, usa código: por ejemplo, `len("fresa")` en Python. La IA puede explicar el procedimiento, pero el programa debe verificar el resultado.

Tres estimaciones útiles — no son leyes exactas

- ▶ Español: como cálculo rápido, **100 palabras ≈ 150 tokens**. Puedes multiplicar palabras × 1,5.
- ▶ Inglés: con frecuencia usa un poco menos, cerca de **100 palabras ≈ 130 tokens**.
- ▶ Una página de unas 500 palabras puede rondar **700 tokens**. El valor real cambia según el idioma, los símbolos y el tokenizador.

La ventana de contexto: el espacio de trabajo del modelo

En palabras simples

La **ventana de contexto** es la cantidad máxima de tokens que el modelo puede usar en una misma interacción. En ese espacio se juntan tus instrucciones, los mensajes anteriores, los archivos enviados y la respuesta que se está generando. Imagina un escritorio: aunque sea grande, no cabe una cantidad infinita de cuadernos.

El tamaño depende del modelo y del servicio: algunos aceptan unas cuantas páginas y otros pueden trabajar con varios libros. **Más contexto no garantiza una mejor respuesta**; también debe estar bien organizado.

¿Qué ocurre cuando se supera el límite? Según la herramienta, puede recortar mensajes antiguos, resumirlos, ignorar parte del contenido o rechazar la solicitud. Por eso un chat largo puede parecer que “olvida”. No es memoria humana: la información dejó de estar disponible en el contexto que el modelo está usando en ese momento.

Tip del sándwich **para textos largos**

Organiza un prompt largo así: **tarea y reglas al inicio → material en el centro → recordatorio del resultado esperado al final**. Además, usa títulos y separadores. Esta estructura reduce confusiones y facilita que tú también revises la instrucción.

Cómo estimar el costo de uso

Una **API** permite que tu programa envíe información a un modelo y reciba una respuesta. Muchos servicios cobran por tokens de **entrada** y de **salida**. Las tarifas cambian con frecuencia, así que la tabla siguiente es solo una referencia didáctica; antes de construir un sistema real debes consultar el precio oficial.

Tipo de modelo (referencia)	Entrada (USD / 1M)	Salida (USD / 1M)
Grande (tareas complejas)	~\$3 – \$15	~\$15 – \$75
Medio (uso general)	~\$1 – \$3	~\$4 – \$15
Pequeño (tareas sencillas)	~\$0.05 – \$0.50	~\$0.25 – \$2

Veamos un ejemplo de aula: resumir un informe de 30 páginas, con unas 15.000 palabras en español. Usaremos tarifas inventadas para practicar el cálculo; no representan una cotización actual.

• • • costo_api.py – tu primer cálculo de costo

```

palabras = 15000
tokens_entrada = int(palabras * 1.5)  # estimación rápida para español
tokens_salida = 1500                  # salida esperada (ejemplo)

precio_in, precio_out = 1.00, 5.00    # USD por millón de tokens
costo = (tokens_entrada/1e6)*precio_in + (tokens_salida/1e6)*precio_out

print(f"1 informe:      ${costo:.4f}")    # → $0.03 (3 centavos)
print(f"10.000 informes: ${costo*10_000:,.0f}") # → $300

```

La lección es doble: una consulta puede costar muy poco, pero miles de consultas forman una factura importante. Diseña instrucciones breves, limita respuestas innecesariamente largas y prueba primero el modelo más económico que cumpla la tarea con la calidad requerida.

Mini-actividad 1.1 — Calcula como desarrollador (4 min)

Usa la estimación del español ($\times 1,5$) y estas tarifas didácticas: entrada USD 1 y salida USD 5 por millón de tokens.

- ▶ a) ¿Cuántos tokens tiene un correo de 200 palabras en español?
- ▶ b) Un chatbot atiende 50.000 consultas al mes. Cada consulta usa cerca de 800 tokens de entrada y 300 de salida. ¿Cuál sería el costo mensual?

Soluciones al final del módulo.

Idea para llevar

Antes de procesar un texto, la IA lo divide en **tokens**. Estos ocupan espacio en la ventana de contexto y pueden generar costo en una API. Para una estimación rápida en español: **palabras $\times 1,5 \approx$ tokens**; para el valor real, usa el contador del modelo.

SECCIÓN

1.2

Embeddings: el significado hecho mapa

La idea que conecta el lenguaje humano con los números



Brújula de la sección — al terminar sabrás:

- ▶ Qué es un embedding mediante la analogía de un mapa.
- ▶ La regla de oro: **cerca en el mapa = parecido en significado**.
- ▶ Cómo comparar dos significados con un valor de similitud.
- ▶ Cuatro aplicaciones que podrás construir con esta idea.

Los **embeddings** son un puente entre el lenguaje y las matemáticas. **Permiten representar una palabra, una frase o un documento mediante números**. Gracias a ellos podemos buscar por significado, recomendar contenidos, clasificar mensajes y recuperar información para un chatbot.



La gran idea: un mapa de significados



¿Qué es un embedding? Piensa en un mapa con coordenadas 📍

En un mapa, las coordenadas permiten ubicar lugares y medir qué tan cerca están. Un **embedding** hace algo parecido: convierte un texto en una lista de números, llamada **vector**, que indica su **posición en un mapa matemático de significados**.

La regla esencial es: **significados parecidos suelen quedar cerca**. “Perro” y “gato” aparecerían en una zona semejante; “perro” y “factura” estarían mucho más separados. El modelo aprende estas relaciones al observar grandes cantidades de ejemplos.

Los sistemas reales usan cientos o miles de números por vector, demasiados para dibujarlos. En este tutorial usaremos solo dos dimensiones inventadas. El dibujo es una simplificación para comprender la idea, no una copia exacta del interior de un modelo.

Construyamos un mapa de práctica con dos preguntas sencillas. Los números son inventados y solo sirven para aprender:

Palabra	¿Qué tan "animal"?	¿Qué tan "realeza"?	Sus coordenadas
gato 🐱	0.9	0.1	[0.9, 0.1]
perro 🐶	0.9	0.0	[0.9, 0.0]
rey 👑	0.1	0.9	[0.1, 0.9]
reina 👑	0.1	1.0	[0.1, 1.0]
tejado 🏠	0.0	0.0	[0.0, 0.0]

Al dibujarlo aparecen grupos o “barrios” de significado:



“Gato” y “perro” quedan cerca porque comparten la característica de animal. “Rey” y “reina” forman otro grupo relacionado con la realeza. En un embedding real, las dimensiones normalmente no tienen nombres tan claros; el entrenamiento descubre patrones mucho más complejos.

Una demostración histórica: relaciones entre vectores

Algunos modelos de palabras mostraron que ciertas relaciones podían aparecer como direcciones en el espacio. El ejemplo histórico más conocido es:

$$\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$$

La operación intenta conservar la idea de “realeza” y cambiar una relación de género. Es una **analogía útil**, pero no funciona de manera perfecta en todos los modelos ni en todas las palabras. También puede reflejar sesgos presentes en los datos de entrenamiento.

Cuidado: representar patrones no es comprender como una persona

El modelo relaciona “perro” y “gato” porque encuentra usos parecidos en los datos, no porque haya visto, cuidado o sentido cariño por un animal. **Un embedding es una representación matemática de patrones.** Puede ser muy útil, pero no posee experiencias humanas ni garantiza que una respuesta sea verdadera.

Comparar significados: medir la cercanía entre vectores

Ya tenemos el mapa; ahora necesitamos medir la cercanía. Una medida común es la **similitud del coseno**. Compara dos vectores y produce un número entre -1 y 1. Para comenzar, basta con recordar: un valor alto suele indicar mayor parecido; un valor bajo, menor relación.

Resultado	Qué significa	Ejemplo
≈ 1.0	Casi el mismo significado	"médico" vs "doctor"
≈ 0.7	Mismo tema, no lo mismo	"médico" vs "hospital"
≈ 0.2	Poco o nada que ver	"médico" vs "guitarra"

... **similitud.py** – la función que usarás mil veces

```
import math
```

```
def similitud(a, b):          # recibe dos vectores, devuelve un número
    punto = sum(x*y for x, y in zip(a, b))
```

```

largo_a = math.sqrt(sum(x*x for x in a))
largo_b = math.sqrt(sum(x*x for x in b))
return punto / (largo_a * largo_b)

gato, perro, rey = [0.9, 0.1], [0.9, 0.0], [0.1, 0.9]
print(similitud(gato, perro))    # 0.99 → vecinos de barrio
print(similitud(gato, rey))     # 0.22 → barrios lejanos

```

Cómo interpretar una similitud sin memorizar umbrales

Los valores de la tabla son orientativos. No existe un límite universal que funcione para todos los modelos y proyectos. Reúne ejemplos reales, compara resultados y elige un umbral que reduzca los errores importantes de tu aplicación.

¿Para qué sirve? Cuatro aplicaciones fáciles de reconocer

Existen modelos especializados que convierten texto, imágenes, audio o código en vectores. Después, una aplicación puede comparar esos puntos y resolver tareas útiles:

Aplicación	La idea en una frase	Ejemplo real
 Búsqueda semántica	Busca los documentos con significado más cercano, aunque usen palabras diferentes.	Buscas “cómo cancelar una compra” y encuentras “política de reembolsos”.
 Recomendación	Recomienda elementos cercanos a los que ya te gustaron.	Canciones, videos o productos relacionados.
 Clasificación	Compara un texto nuevo con categorías conocidas y elige la más cercana.	Enviar una solicitud al equipo adecuado.
 Memoria de chatbots (RAG)	Guarda documentos como vectores y recupera los más relacionados con cada pregunta.	Un chatbot que consulta apuntes, manuales o documentos autorizados.

Un buscador por palabras puede fallar cuando la consulta dice “cancelar” y el documento dice “reembolso”. La búsqueda semántica compara significado y puede acercar ambas expresiones. Recuerda: **buscar semánticamente es encontrar vecinos en un mapa matemático.**

 Mini-actividad 1.2 — El cartógrafo (5 min)

Responde primero con tu intuición, sin usar la fórmula:

- ▶ **a)** Ordena de mayor a menor similitud: (médico, doctor) · (médico, hospital) · (médico, guitarra).
- ▶ **b)** En nuestro mini-mapa, ¿qué coordenadas aproximadas le darías a "princesa"? ¿Y a "ratón"?
- ▶ **c)** Un cliente busca "el paquete llegó roto". ¿Qué documento recuperará la búsqueda vectorial: "Política de devoluciones por daños" o "Horarios de atención"? ¿Por qué, si no comparten palabras?

Soluciones al final del módulo.

 Idea para llevar

Un **embedding** representa un contenido con un vector. Los contenidos parecidos suelen quedar cerca y una medida de similitud ayuda a compararlos. Así, buscar, recomendar, clasificar y recuperar información pueden verse como una misma tarea: **encontrar vecinos**.

SECCIÓN

1.3**Cómo produce una respuesta (sin fórmulas)**

Cómo usa el contexto y elige el siguiente token

 Brújula de la sección — al terminar sabrás:

- ▶ Cómo el contexto ayuda a interpretar palabras ambiguas como “banco”.
- ▶ Cómo se genera una respuesta: calculando probabilidades para el siguiente token.
- ▶ Qué es la temperatura y cómo cambia la variedad de las respuestas.
- ▶ Tres consecuencias prácticas que verás cada vez que uses un chatbot.

Ya sabes que el texto se divide en tokens y se representa con números. Ahora veremos cómo un modelo relaciona esas piezas para producir una respuesta. Muchos modelos modernos usan una arquitectura llamada **Transformer**. Aquí estudiaremos dos ideas sencillas: **atención** para usar el contexto y **probabilidades** para elegir el siguiente token.

Idea 1 — La atención: cada token consulta el contexto

En palabras simples

La **atención** permite que cada token dé más importancia a unas partes del texto que a otras. No “mira” de forma humana: calcula relaciones numéricas. Gracias a esto, una palabra puede ajustar su representación según las palabras que la rodean.

El caso de “banco”: una palabra, dos significados

Lee estas dos frases:

- ▶ "Fui al **banco** a retirar dinero." 💰
- ▶ "Me senté en el **banco** de la plaza." 🪑

La palabra “banco” aparece en las dos frases, pero el contexto cambia. En la primera se relaciona con “retirar” y “dinero”; en la segunda, con “senté” y “plaza”. La atención ayuda a construir una representación diferente para cada caso.

En palabras sencillas, la atención adapta el significado al contexto de la frase. Este cálculo se repite en varias capas y puede relacionar información cercana o lejana. Por ahora, quédate con esta pregunta: **¿qué palabras ayudan a interpretar esta palabra?**

Idea 2 — Probabilidades para elegir el siguiente token

Con el contexto disponible, el modelo asigna una probabilidad a muchos posibles tokens de continuación. Puedes imaginar una ruleta: las opciones más probables ocupan una porción mayor. Los porcentajes del ejemplo son inventados para explicar la idea.

Contexto: "El gato se subió al ____"



Temperatura: una perilla para variar la respuesta

Algunas APIs permiten ajustar un parámetro llamado **temperatura**, que modifica cuánta variedad se acepta entre las opciones probables:

- ▶ Temperatura baja → favorece las opciones más probables. Suele convenir para extraer datos, clasificar o seguir un formato. Reduce la variación, aunque no siempre garantiza respuestas idénticas.
- ▶ Temperatura alta → permite más opciones. Puede servir para ideas y escritura creativa, pero aumenta el riesgo de respuestas poco precisas o extrañas.

Regla práctica: **baja para precisión y formato; más alta para variedad creativa.**

El modelo selecciona un token, lo añade al texto y vuelve a calcular la continuación usando el nuevo contexto. Repite el ciclo hasta terminar. Por eso la respuesta aparece poco a poco. También explica por qué una salida larga requiere más tiempo y, en muchos servicios, más costo.

Tres consecuencias prácticas

- ▶ Las respuestas largas suelen tardar y costar más porque contienen más tokens generados.
- ▶ Una respuesta fluida puede contener errores: el modelo produce continuaciones probables, no consulta automáticamente una base de datos de hechos.
- ▶ La misma pregunta puede producir respuestas distintas. Por eso debes probar, comparar y verificar la información importante.

Cuidado: contexto, historial y aprendizaje no son lo mismo

Durante una conversación, el modelo usa el **contexto** disponible para responder, pero eso no significa que sus pesos se reentrenen en ese instante. Una aplicación puede guardar el historial o usar funciones de memoria por separado; no confundas esas funciones con el aprendizaje del modelo base.

Mini-actividad 1.3 — Detective de atención (3 min)

Para cada frase, identifica qué palabras aportan las pistas más importantes para interpretar el término destacado:

- ▶ a) "La **copa** se rompió cuando brindamos con vino."
- ▶ b) "María le dijo a Ana que **ella** había ganado el premio." — ¿Se puede resolver? ¿Qué le falta a la frase?
- ▶ c) "El gato se subió al tejado porque el perro **lo** perseguía." — ¿a quién se refiere "lo"?

Soluciones al final del módulo.

Idea para llevar

Dos ideas esenciales: 1) la **atención** relaciona cada token con el contexto; 2) la respuesta se construye token a token a partir de probabilidades. La temperatura, cuando está disponible, modifica la variedad de esa selección.

SECCIÓN

1.4

Prompts profesionales

Pasa de preguntar al azar a diseñar instrucciones claras

Brújula de la sección — al terminar sabrás:

- ▶ Una receta de 6 ingredientes para organizar un prompt.
- ▶ Few-shot: enseñar la tarea mediante ejemplos.
- ▶ Procedimiento verificable: cuándo dividir una tarea en pasos.
- ▶ Salida JSON: cómo entregar datos que tu programa pueda validar.

Ahora puedes pasar de una pregunta improvisada a una instrucción diseñada. Un buen **prompt** explica qué tarea debe realizarse, con qué información, bajo qué reglas y cómo debe entregarse el resultado. No garantiza perfección, pero hace que el comportamiento sea más claro y más fácil de evaluar.

El cambio de mentalidad (la idea central de la sección)

Una persona puede preguntar cuando algo no está claro. Un modelo, en cambio, intentará continuar con la información disponible. Si no defines el formato, la longitud o qué hacer cuando falta un dato, puede tomar una decisión distinta a la que esperabas.

Regla de oro: **describe qué significa una respuesta correcta**. La ambigüedad se convierte en errores difíciles de detectar. 🐛

La receta de 6 ingredientes

Usa estos bloques como una lista de comprobación. No siempre necesitarás los seis, pero sí debes incluir los que ayuden a evaluar la tarea:

Ingrediente	Qué define	Ejemplo
1. Rol 🤖	La perspectiva o especialidad útil para la tarea; es opcional.	"Actúa como analista de datos de una aseguradora."
2. Tarea 🎯	La acción concreta: clasificar, extraer, comparar, resumir...	"Extrae los datos del siniestro del correo."
3. Contexto 📄	La información necesaria para realizar la tarea.	El correo del cliente, entre etiquetas <correo>...</correo>
4. Formato 📦	La forma esperada: tabla, lista, JSON, extensión, idioma...	"Responde ÚNICAMENTE con JSON con este esquema..."
5. Restricciones ⚠️	Límites, valores permitidos y qué hacer cuando falta información.	"Si falta un dato, usa null. No inventes valores."
6. Ejemplos 📝	Ejemplos completos de entrada y respuesta esperada.	De 2 a 5 ejemplos, incluido un caso difícil o ambiguo.

🔧 Separa las instrucciones del material que se debe analizar

Cuando incluyas un correo, un texto o código, sepáralo con etiquetas como `<documento>...</documento>`. Esto ayuda a distinguir la orden del contenido. Aun así, no envíes información personal, contraseñas o documentos confidenciales sin autorización y sin conocer las políticas de la herramienta.

🎯 Técnica 1 — Few-shot: muestra ejemplos de la respuesta esperada

💡 En palabras simples

En **few-shot** muestras varios ejemplos de entrada y salida antes del caso nuevo. El modelo usa ese patrón para imitar el formato, las categorías y el tono. Los ejemplos son especialmente útiles cuando una regla escrita podría interpretarse de varias maneras.

... prompt_fewshot.txt – un clasificador funcionando en 20 segundos

Clasifica cada reseña como POSITIVA, NEGATIVA o NEUTRA.

Reseña: "Llegó rápido y funciona perfecto"
Clasificación: POSITIVA

Reseña: "Se dañó a la semana, no lo recomiendo"
Clasificación: NEGATIVA

Reseña: "Es tal cual la foto, envío normal"
 Clasificación: NEUTRA

Reseña: "Cumple, aunque esperaba mejor calidad por el precio"
 Clasificación: # ← el modelo continúa el patrón

Reglas para crear buenos ejemplos

- ▶ Asegúrate de que ejemplos e instrucciones coincidan; una contradicción confunde al modelo.
- ▶ Incluye un caso difícil resuelto según el criterio que quieres enseñar.
- ▶ Representa todas las categorías importantes y mantén el mismo formato.
- ▶ Usa pocos ejemplos, pero bien elegidos: también ocupan tokens en el contexto.

Técnica 2 — Define un procedimiento que pueda comprobarse

En palabras simples

En tareas con varias operaciones, indica el **procedimiento visible** que deseas: por ejemplo, “calcula primero el descuento, después el subtotal y al final el impuesto”. Pedir pasos verificables ayuda a revisar el resultado. No confundas una explicación bien escrita con una garantía de verdad: las operaciones importantes deben comprobarse con una calculadora o con código. 

El mismo problema, con y sin procedimiento

Sin procedimiento: “Aplica 15 % de descuento a \$84.000 y después suma IVA del 19 %”. Una respuesta directa es difícil de revisar si aparece un error.

Con procedimiento: “Muestra: 1) descuento, 2) precio descontado, 3) IVA y 4) total. Termina con RESULTADO: <valor>”. Así puedes verificar: \$12.600 de descuento → \$71.400 de subtotal → \$13.566 de IVA → **RESULTADO: \$84.966.**

¿Cuándo usarlo?

Úsalo para cálculos, comparaciones y procesos con un orden obligatorio. No suele hacer falta para extraer un dato sencillo. Pide una explicación breve o una tabla de comprobación, y valida el resultado con una herramienta adecuada.



Técnica 3 — Salida JSON: entrega datos que tu programa pueda leer

Cuando otro programa debe leer la respuesta, conviene usar un formato estructurado como **JSON**. El programa espera claves y valores exactos; un saludo o una explicación adicional puede causar un error. Sigue estas reglas:

1. Indica una salida cerrada: “Responde únicamente con JSON válido; no agregues saludos, explicaciones ni bloques de Markdown”.
2. Muestra el esquema: nombres de las claves, tipos de datos y valores permitidos.
3. Define los datos ausentes: “si no aparece, usa null”. Nunca pidas que complete información que no está en la fuente.
4. Incluye al menos un ejemplo completo de entrada y JSON correcto.
5. Si la API ofrece **salidas estructuradas** o un esquema, utilízalo. En cualquier caso, el programa debe validar el JSON, los tipos y los valores antes de guardarlos o usarlos.

• • • prompt_extractor.txt – de correo caótico a datos limpios

Eres un sistema de extracción de datos de una empresa de logística.
 Extrae los datos del correo y responde ÚNICAMENTE con JSON válido.
 Esquema (usa null si un dato no aparece; no inventes valores):

```
{
  "cliente": "texto",
  "numero_pedido": "texto o null",
  "problema": "dañado | retrasado | perdido | otro",
  "urgente": true | false
}
```

<correo>

Buenas, soy Marta Ríos. El pedido A-4412 llegó con la caja destrozada y necesito solución HOY porque es un regalo.

</correo>

Salida del modelo (lista para tu programa):

```
# {"cliente": "Marta Ríos", "numero_pedido": "A-4412",
#  "problema": "dañado", "urgente": true}
```

⚠ Cuidado: no existe un prompt perfecto ni una respuesta infalible

Un prompt funciona bien solo después de probarlo con casos normales, incompletos y difíciles. Registra los errores, mejora las instrucciones y vuelve a probar. Además, verifica los datos importantes y evita enviar información sensible. Un prompt sin pruebas es una idea; no todavía una solución confiable.

 Mini-actividad 1.4 — Cirugía de prompt (5 min)

Este prompt es confuso: “dime qué opinan los clientes de estos comentarios y si son buenos o malos ponlo en una lista o algo así, gracias”.

Encuentra al menos cuatro problemas y reescríbelo con tarea, categorías, contexto delimitado, reglas y salida JSON. *Solución al final del módulo.*

 Idea para llevar

Un buen prompt define **tarea + contexto + formato + restricciones + ejemplos**; el rol es opcional cuando aporta una perspectiva útil. Los ejemplos enseñan el patrón, un procedimiento permite comprobar tareas complejas y el JSON conecta la respuesta con código que debe validarla.

SECCIÓN

**Evaluación práctica**

Tres retos para aplicar lo aprendido con productos concretos

Los retos 1 y 2 pueden resolverse con papel, calculadora o Python. Para el reto 3 puedes usar un chatbot, pero trabaja con textos inventados o sin datos personales. Guarda cada versión: comparar el antes y el después también forma parte del aprendizaje.

 RETO 1 — El presupuestador de tokens · **45 min** · **Dificultad:**

Objetivo: practicar la estimación de tokens y costos con un caso sencillo de empresa.

Escenario: una empresa quiere resumir 2.000 reclamaciones al mes. Cada texto tiene cerca de 600 palabras en español; cada resumen, 100 palabras. Además, cada consulta incluye 300 tokens de instrucciones.

Pasos:

1. Convierte las palabras a tokens con la estimación $\times 1,5$. Suma los 300 tokens de instrucciones en cada consulta.
2. Calcula el costo mensual con estas tarifas didácticas: modelo A (entrada USD 1, salida USD 5) y modelo B (entrada USD 10, salida USD 40), por millón de tokens.
3. Calcula el costo anual de ambas opciones y la diferencia.
4. Escribe una recomendación de cinco líneas. Compara costo, calidad necesaria y cantidad de solicitudes. Elige la opción más económica que cumpla el objetivo.

Entregable: una página con los cálculos visibles (se permite redondear) y la recomendación.

Criterio de éxito: procedimiento visible, unidades correctas y una recomendación apoyada en los resultados.

🏆 RETO 2 — El cartógrafo semántico · ⌚ 45 min · Dificultad: ★★★★★

Objetivo: comprender los embeddings mediante un mapa de significados creado por ti.

Pasos:

1. Elige un dominio que te guste (cocina, fútbol, música, videojuegos...) y selecciona 10 palabras, incluyendo al menos 2 "intrusas" de otro tema.
2. Inventa dos dimensiones fáciles de explicar, asigna coordenadas entre 0 y 1 y dibuja el mapa. Escribe que los datos son una simplificación educativa.
3. Con la función similitud() de la sección 1.2 (o calculadora), calcula la similitud de 3 pares: uno muy parecido, uno intermedio y uno con una intrusa.
4. Compara los números con tu intuición. Si no coinciden, revisa las coordenadas y explica qué aprendiste del error.

Entregable: foto del mapa + tabla de coordenadas + los 3 resultados con su interpretación.

Criterio de éxito: cálculos correctos y una explicación propia de por qué una búsqueda semántica compara cercanía entre vectores.

🏆 RETO 3 — Tu primer prompt de producción · ⌚ 60–90 min · Dificultad: ★★★★★

Objetivo: diseñar, probar y mejorar un prompt con ejemplos y salida JSON.

Pasos:

1. Elige una tarea repetitiva: clasificar mensajes ficticios, organizar apuntes o etiquetar gastos inventados. No uses datos privados.
2. Escribe la versión 1 con los 6 ingredientes: rol, tarea, contexto entre etiquetas, formato JSON con esquema, restricciones (incluido "dato ausente → null") y 3 ejemplos few-shot (uno difícil).
3. Pruébalo con cinco casos: tres normales y dos difíciles, como un dato ausente, un texto ambiguo o una categoría no prevista.
4. Anota qué falló, crea una versión 2 y repite la prueba. Después prepara una versión 3 con la mejora más importante.

Entregable: las 3 versiones del prompt + tabla de resultados + 5 líneas sobre qué cambio mejoró más los resultados.

Criterio de éxito: la versión 3 produce JSON válido en al menos cuatro de los cinco casos, usa null cuando corresponde y no agrega texto externo.

SECCIÓN



Material de referencia

Glosario esencial, soluciones y autoevaluación final

Glosario esencial (12 términos)

Repasa estas palabras al día siguiente y explícalas con un ejemplo propio. Recordarás mejor lo que puedes enseñar.

Término	En una frase
Token	Pieza de texto producida por el tokenizador: puede ser una palabra, una parte, un número o un signo.
Vocabulario	Conjunto de tokens que un modelo puede reconocer directamente.
Ventana de contexto	Límite de tokens disponibles para instrucciones, historial, documentos y respuesta.
Embedding	Representación numérica de un contenido en un espacio donde lo parecido suele quedar cerca.
Vector	Lista ordenada de números que representa una posición en el espacio matemático.
Similitud del coseno	Medida entre -1 y 1 que compara la orientación de dos vectores; valores altos suelen indicar más similitud.
Búsqueda semántica	Recuperar contenidos por parecido de significado, aunque no usen las mismas palabras.
Transformer	Arquitectura que usa atención para relacionar tokens y construir representaciones de contexto.
Atención	Cálculo que asigna distinta importancia a otras partes del contexto.
Generación autorregresiva	Generar un token, añadirlo al contexto y repetir el proceso para obtener el siguiente.
Temperatura	Parámetro que controla la variedad: bajo favorece respuestas más estables; alto permite más diversidad.
Few-shot	Mostrar varios ejemplos de entrada y salida para enseñar el patrón esperado.

✓ Soluciones de las mini-actividades

Mini-actividad 1.1 (presupuesto de tokens)

- ▶ a) $200 \text{ palabras} \times 1,5 \approx 300 \text{ tokens}$. Es una estimación.
- ▶ b) Entrada: $50.000 \times 800 = 40 \text{ millones} \rightarrow 40 \times \text{USD } 1 = \text{USD } 40$. Salida: $50.000 \times 300 = 15 \text{ millones} \rightarrow 15 \times \text{USD } 5 = \text{USD } 75$. **Total aproximado: USD 115 al mes.**

Mini-actividad 1.2 (el cartógrafo)

- ▶ a) (médico, doctor) > (médico, hospital) > (médico, guitarra): casi sinónimos > tema relacionado > temas distintos.
- ▶ b) Una respuesta posible: princesa $\approx [0.1, 0.9]$ y ratón $\approx [0.9, 0.0]$. Como las coordenadas son inventadas, otras respuestas también pueden ser coherentes si se justifican.
- ▶ c) “Política de devoluciones por daños”. “Roto” se relaciona con “daños” y el contexto de una entrega defectuosa se relaciona con una devolución. La búsqueda compara el significado general, no solo palabras idénticas.

Mini-actividad 1.3 (detective de atención)

- ▶ a) “Copa” se relaciona con “brindamos” y “vino”; por eso significa recipiente para beber, no trofeo.
- ▶ b) No puede resolverse con seguridad: “ella” podría ser María o Ana. La atención usa pistas existentes; no debería inventar la que falta.
- ▶ c) “Lo” se refiere al gato, porque el perro es quien lo perseguía.

Mini-actividad 1.4 (cirugía de prompt)

- ▶ Problemas posibles: tarea imprecisa, categorías poco claras, ausencia de contexto delimitado, formato indefinido, falta de reglas para comentarios ambiguos y ausencia de un ejemplo.
- ▶ Reescritura posible: “Clasifica cada comentario como POSITIVO, NEGATIVO o NEUTRO. Analiza únicamente el texto entre <comentarios>...</comentarios>. Responde solo con JSON válido: {"resultados": [{"comentario": "texto", "sentimiento": "POSITIVO | NEGATIVO | NEUTRO"}]}. Si no hay información suficiente, usa NEUTRO”. Añade dos o tres ejemplos coherentes.



Cierre: el recorrido completo de la frase del gato

Repasa el recorrido: “El gato se subió al tejado” puede dividirse en varios tokens (1.1); esos elementos se representan con números y relaciones de significado (1.2); la atención usa el contexto de la frase (1.3); y un prompt claro orienta la tarea y el formato de la respuesta (1.4).

Ya no necesitas llamarlo magia: ahora reconoces un sistema que puedes orientar, medir y verificar. 🔧

**Autoevaluación final — marca solo lo que puedas hacer SIN mirar el módulo**

- ▶ ☐ Explicar qué es un token, por qué depende del tokenizador y por qué conviene verificar tareas con caracteres.
- ▶ ☐ Estimar los tokens de un texto en español ($\times 1,5$) y calcular un costo didáctico de API.
- ▶ ☐ Definir embedding con la analogía del mapa e interpretar valores altos y bajos de similitud.
- ▶ ☐ Explicar cómo el contexto aclara “banco” y cómo la temperatura modifica la variedad.
- ▶ ☐ Escribir un prompt claro, con ejemplos, restricciones, salida JSON y una forma de validarla.

¿Puedes marcar las cinco y explicar tus decisiones en los retos? Entonces estás listo para convertir estas ideas en código. 🎓